EC8202 Big Data Analytics

Smart Grid Energy Monitoring & Billing

A Lambda Architecture Data Pipeline Use Case 3

Group Members

EG/2020/4351 — Perera M.T.T.M
EG/2020/4202 — Senarathna G.M.L.D
EG/2020/4128 — Rajakaruna R.H.M.K.M

Table of Contents

1. Introduction & Use Case Selection

2. Business Requirements

3. Architecture Decision: Lambda vs. Kappa

4. Technology Stack & Justification

5. System Design

6. Data Sources & Simulated Clock

7. Processing Layer

8. Storage & Serving Layer

9. Observability Design

10. Results

11. Testing

12. Limitations, Trade-offs & Production-Scale Changes

13. Conclusion

14. Individual Contributions

1. Introduction & Use Case Selection

This report presents the design and development of a complete data engineering pipeline for the EC8202 Big Data Analytics mini-project. Among the three available scenarios, Use Case 3-Smart Grid Energy Monitoring & Billing was selected for implementation.

The selected use case focuses on the requirements of a utility company that needs both real-time monitoring of electricity consumption and solar energy contribution from smart meters across different service zones, as well as the generation of accurate daily household bills once tariff and billing reference data becomes available. Smart meter readings represent a continuous and high frequency stream of data, while tariff, billing, and weather information are generally processed as periodic or daily datasets.

Since the system must handle both real-time streaming data and scheduled batch data, a Lambda architecture provides a suitable solution. This architecture enables the integration of fast stream processing with reliable batch processing, making the selected use case well suited for demonstrating the key concepts and learning outcomes of the module.

Why this use case over the alternatives: Although the Ride-Hailing Fleet Operations and Hospital Patient Vital Signs Monitoring scenarios also involve a combination of real-time streaming and daily batch processing, the Smart Grid scenario was selected because its business requirements can be divided into two clear and measurable outputs: the real-time percentage of renewable energy contribution and the daily household electricity bill. These outputs are straightforward to calculate and verify, making the overall processing logic easier to understand, test, validate, and explain during the project viva.

2. Business Requirements

Translating the scenario into practical system requirements:

- Continuously collect smart-meter readings for each household, including power consumption (kWh) and solar generation (kWh), together with the relevant grid zone.
- Process a daily tariff and billing reference dataset containing information such as household ID, tariff rate, billing tier, and subsidy status. A daily weather forecast dataset, including grid zone and cloud-cover information that may influence solar generation, is also ingested.
- Provide a near-real-time view of the current electricity demand and renewable-energy contribution for each grid zone, with updates available within less than a minute.
- Perform a daily batch calculation to determine each household's electricity bill after applying the latest tariff information to its recorded consumption. The same process also calculates the total solar-energy contribution for each grid zone. Since this computation is based on stored data, it can be reproduced or reprocessed when necessary.
- Automatically generate alerts when the renewable-energy contribution of a grid zone falls significantly below the defined threshold or when the pipeline detects that incoming smart-meter data has stopped.

These requirements correspond directly to the three main outputs suggested in the project brief: a real-time API for grid load and renewable-energy monitoring, automated threshold-based alerts, and a daily consolidated report containing household billing and solar-contribution information. All three outputs are implemented in this project and are discussed further in Sections 8–10.

3. Architecture Decision: Lambda vs. Kappa

3.1 The candidates

Kappa architecture processes all data through a single streaming pipeline. A stream-processing framework such as Spark Structured Streaming or Flink reads data from an append-only, replayable log. Instead of maintaining a separate batch-processing system, historical or batch-style results are generated by replaying the stream from a selected point in the log.

In contrast, Lambda architecture uses two separate processing paths. The speed layer handles incoming data to provide low-latency, near-real-time results, while the batch layer periodically reprocesses the complete immutable dataset to produce accurate and authoritative results. A serving layer then combines the outputs of both layers to provide the final view of the data.

3.2 Why Lambda fits this use case better

Different latency requirements for each output:
The two main business outputs require different levels of speed and accuracy. The current grid load by zone only needs to provide a reasonably accurate result within about a minute, so a small amount of delay, duplication, or missing data can be tolerated. In contrast, the household billing result must be precise, traceable, and reproducible because it directly relates to financial calculations. Lambda architecture supports these different requirements by allowing the speed layer to focus on low-latency approximate results, while the batch layer follows stricter accuracy and consistency requirements. With Kappa architecture, both outputs would need to use the same stream-processing guarantees, which could make the real-time view unnecessarily complex or make the billing process less robust.

Billing is naturally based on a periodic reference dataset:
The billing calculation requires smart-meter consumption data to be joined with tariff and billing information that is updated only once per day. This reference data behaves more like a batch dataset than a continuous stream. Kappa architecture is most suitable when all major inputs are naturally represented as continuous streams. Converting the daily tariff file into a Kafka topic only to maintain a single streaming pipeline would introduce extra complexity, including another topic and additional stream-table join logic. Lambda architecture handles this situation more naturally because its batch layer is designed to work with periodically arriving reference data.

Replay and correction requirements favour Lambda architecture:
The system must also support reprocessing an entire day's data to generate an authoritative billing result. This is important when late-arriving readings, duplicate events, temporary message loss, or processing errors affect the real-time calculations. Lambda architecture directly supports this behaviour because the batch layer can recompute results from the complete stored dataset and correct inaccuracies produced by the speed layer. Although Kappa architecture can also replay streams, it depends heavily on sufficient event-log retention and the continued availability of the original source data. Maintaining long retention

periods for reliable daily reprocessing can reduce some of the simplicity and cost advantages normally associated with Kappa.

Alignment with the assignment technology stack:
The recommended technology stack also fits naturally with a Lambda-style design. Kafka together with Spark Structured Streaming or Storm is intended for the real-time processing component, while Airflow is used separately to coordinate batch jobs and reporting workflows. This separation already reflects the main structure of Lambda architecture. In the implemented system, Airflow manages the independent daily pipeline, including file availability checks, retries, data-quality validation, and report generation. In a pure Kappa design, Airflow would mainly be used to trigger stream replays rather than manage a separate batch-processing workflow.

3.3 Rejected alternative and its trade-offs

Kappa architecture was also considered as an alternative because it could simplify the overall implementation. It would remove the need for a separate batch-processing script such as **`spark/batch_job.py`**, reduce the need to maintain both raw and aggregated storage paths, and avoid one of the common challenges of Lambda architecture: keeping two separate processing pipelines consistent.

However, Kappa architecture was not selected for several reasons:

- It would require Kafka to retain at least one complete simulated day of smart-meter events, and preferably longer to provide a safety margin. This would increase storage requirements and operational complexity, which is unnecessary for a small, two-week project running on a single Kafka broker.
- The tariff and billing information is naturally a daily reference dataset, rather than a continuous sequence of events. Representing each household's daily tariff information as Kafka events would therefore add complexity without providing a clear architectural benefit.
- Kappa would also make the billing process less straightforward to audit and reproduce. In the Lambda implementation, the **`household_bills`** result for a particular **`sim_date`** can be regenerated deterministically from the immutable Parquet data together with the corresponding daily CSV reference file. This makes it easy to rerun, compare, and verify billing results when necessary. Achieving the same behavior through Kappa would require careful management of stream offsets, checkpoints, and replay boundaries to reproduce an identical billing calculation.

The implementation also reduces the usual Lambda concern of maintaining duplicate business logic. Although **`spark/streaming_job.py`** and **`spark/batch_job.py`** are separate Spark applications, both use the same Spark DataFrame expression approach for calculations such as renewable-energy contribution and billing-related values. The core calculation formula is defined consistently and unit-tested in **`sources/simulation_logic.py`** (see section 11). Therefore, most of the duplication exists at the infrastructure level, such as separate Spark jobs and output paths, rather than in the underlying business logic.

4. Technology Stack & Justification

Layer	Choice	Justification tied to this use case
Ingestion	Apache Kafka (6 partitions, key = grid_zone)	The architecture separates the smart-meter simulator from Spark, allowing the streaming application to be restarted independently without losing previously generated events. In addition, partitioning the data by `grid_zone` allows Spark to process zone-based window aggregations in parallel, improving processing efficiency and reducing unnecessary data movement between tasks.
Stream processing	Spark Structured Streaming	Spark provides built-in support for window-based aggregations, including both tumbling and sliding windows, together with checkpointing and exactly-once micro-batch processing semantics. Using Spark for both the streaming and batch layers also simplifies the overall architecture. Rather than combining Storm for real-time processing with Spark for batch processing, a single processing framework is used throughout the system, reducing implementation complexity and allowing the same DataFrame-based calculation logic to be maintained consistently across both layers.
Orchestration	Apache Airflow	The batch-processing layer requires several workflow-management capabilities, including automatic retries, verification that the required input files are available before execution through a **`FileSensor`**, post-processing data-quality validation, and a clear execution history for auditing daily financial calculations. These requirements align closely with Airflow's core strengths in workflow orchestration and monitoring, and they are separate from the continuous data-processing responsibilities handled by Kafka and Spark.
Storage — raw/master	Parquet on a shared volume, partitioned by sim_date + grid_zone	The batch layer uses a low-cost, column-oriented storage format that also supports reliable data replay. This stored dataset acts as the immutable master copy from which **`household_bills`** can be recalculated whenever necessary, without depending on Kafka's comparatively shorter message-retention period.
Storage/Sink — serving	PostgreSQL	The serving layer requires fast indexed lookups using fields such as **`household_id`**, **`grid_zone`**,

Layer	Choice	Justification tied to this use case
		and **`sim_date`**, together with lightweight aggregate queries. A relational database with primary keys and B-tree indexes can handle these operations efficiently with very low query latency. Although Cassandra is well suited to systems that require extremely high write throughput across many distributed partitions, that capability is unnecessary at the scale of this simulation. Therefore, PostgreSQL was selected because it offers a simpler design and provides familiar SQL-based querying.
Serving API	FastAPI + Uvicorn	An asynchronous and lightweight REST framework was selected to minimise development complexity while providing built-in OpenAPI documentation through the **`/docs`** interface. This makes it easy for both evaluators to inspect and test the available API endpoints and for the dashboard to integrate with them. In addition, Pydantic-based validation ensures that request and response structures are clearly defined and consistently enforced.
Observability	Structured (JSON) logging in every component + prometheus-client + Airflow's own run history/alerting	Structured JSON logging allows logs from the different components of the system, including Python services, PySpark, Airflow, and FastAPI, to be processed and analysed consistently through **`docker compose logs`**. In addition, Prometheus counters and gauges expose system metrics through a standard **`/metrics`** endpoint, allowing the application to be integrated with monitoring tools in a real deployment without requiring significant code changes.

Two intentional simplifications were made to the preferred technology stack and are stated explicitly as required by the project brief. First, HDFS or S3 was not used for the raw data archive. Instead, a local Docker volume storing Parquet files was selected because the system is demonstrated on a single node and therefore does not require a distributed file system. The implementation can still be migrated easily to S3 or HDFS at production scale, since the existing **`spark.read.parquet(path)`** approach only requires the storage path to be changed.

Second, Cassandra was evaluated as an option for the serving layer, but PostgreSQL was selected because its relational model, indexing capabilities, and simpler SQL-based querying were more suitable for the scale and access patterns of this project.

5. System Design

Figure 1 illustrates the complete data-processing pipeline. Two simulated data sources provide information through Kafka for real-time streaming and through a shared storage volume for daily batch processing. In the speed layer, Spark Structured Streaming continuously processes and aggregates incoming meter data, stores the resulting real-time values in PostgreSQL, and archives the raw events in Parquet format.

For the batch layer, Airflow schedules and manages processing once per simulated day. The batch job combines the stored Parquet data with the corresponding tariff and weather CSV files, then writes the final validated results to the **`household_bills`** and **`zone_solar_daily`** tables. The FastAPI serving layer provides access to both the real-time and batch results stored in PostgreSQL through a unified REST API. The dashboard queries this API every five seconds to display updated information to the user.

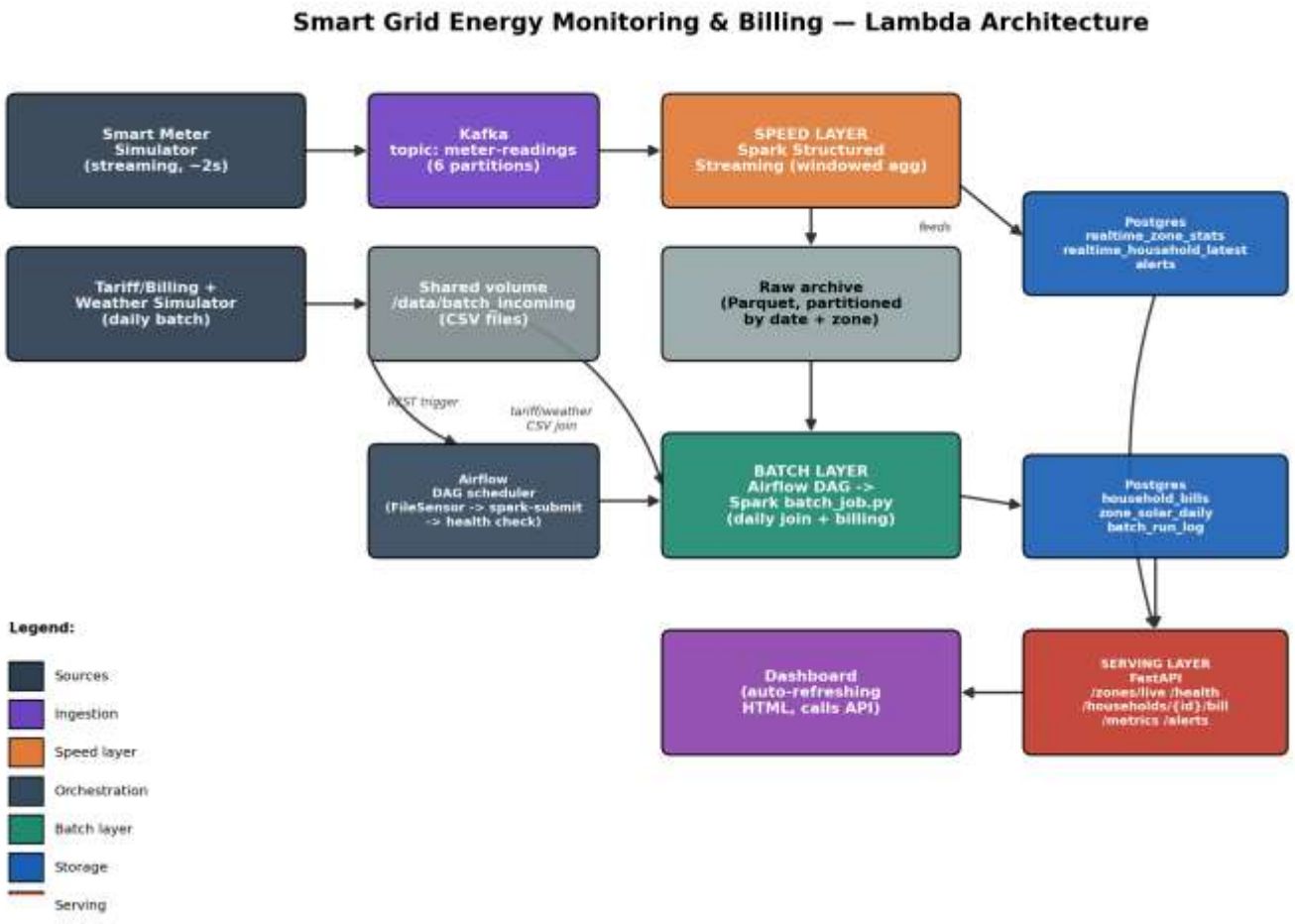


Figure 1 — End-to-end Lambda architecture: ingestion, speed layer, batch layer (Airflow-orchestrated), storage, and serving layer.

5.1 Repository layout

The codebase is organized by pipeline layer so each rubric criterion maps onto one directory:

- sources/ — simulated data sources (streaming meter simulator, daily tariff/weather batch source) + pure billing/generation logic used by both the simulators and the unit tests.

- spark/ — streaming_job.py (speed layer) and batch_job.py (batch layer).
- airflow/dags/ — smart_grid_batch_dag.py, the daily orchestration DAG.
- api/ — FastAPI serving layer, Postgres schema (schema.sql), and DB access helpers.
- dashboard/ — the single-page live dashboard.
- tests/ — pytest unit tests for the billing formula and the simulators.
- docker-compose.yml — brings up all nine services with one command (Kafka, Zookeeper, Postgres, Spark master/worker, the streaming job, Airflow webserver+scheduler, the API, and both simulators).

6. Data Sources & Simulated Clock

6.1 Streaming source — sources/meter_simulator.py

Simulates 20 smart meters (5 households across each of 4 grid zones, a mix of billing tiers, ~1/3 with rooftop solar) publishing one JSON reading per household to the meter-readings Kafka topic every 2 seconds. Solar output is modelled as a function of a compressed "time of day" (a sine curve peaking mid-cycle) and a slowly-drifting cloud-cover value, so the low-renewable alert has something realistic to occasionally trip on independent of the daily weather file. Every reading carries meter_id, household_id, power_consumption_kwh, solar_generation_kwh, grid_zone and timestamp, matching the brief's schema.

6.2 Daily-batch source — sources/tariff_batch_source.py

Once per simulated day, writes tariff_<YYYYMMDD>.csv (household_id, tariff_rate, billing_tier, subsidy_flag) and weather_<YYYYMMDD>.csv (grid_zone, date, cloud_cover, forecast_summary) to a shared volume, then calls the Airflow REST API to trigger that day's DAG run for the corresponding logical date — modelling a vendor file drop that kicks off a downstream job, as a production system would see via SFTP or a webhook.

6.3 Simulated clock

1 simulated day = 300 seconds (5 minutes) of wall-clock time: (SIM_DAY_SECONDS in sources/common.py, overridable via .env). This was chosen so that at least two full day-cycles — enough to see the batch layer run twice and compare consecutive daily bills — can be observed within a single ~10–15 minute demo session, satisfying the brief's request to state and justify any time compression used.

Both simulators derive their notion of "day" from the same sim_day_index()/sim_date_str() helpers in sources/common.py, so the streaming archive's date partitions and the batch file's date suffix always line up — this alignment is what makes the batch job's join correct.

7. Processing Layer

7.1 Speed layer — spark/streaming_job.py

Consumes meter-readings continuously and performs three concurrent Structured Streaming queries against the same parsed, cleaned stream (nulls dropped, negative readings clamped to zero — the "not just pass-through" cleaning step):

- Raw archive: every cleaned event is appended to Parquet, partitioned by sim_date and grid_zone — this is the batch layer's master dataset.
- Windowed zone aggregation: a 1-minute tumbling window with a 30-second slide, grouped by grid_zone, summing consumption and solar generation and computing renewable_share; written via a foreachBatch sink that upserts into Postgres (ON CONFLICT) so realtime_zone_stats always reflects the latest windows without unbounded growth of stale rows. Whenever a window's renewable share drops below LOW_RENEWABLE_THRESHOLD (10%), a WARNING-level alert row is inserted — the streaming half of the Observability requirement's alert rule.
- Latest-reading-per-household: an append-mode stream upserting realtime_household_latest, keyed by household_id, guarded so only strictly newer readings overwrite older ones — this both powers the household "live" view and is the freshness signal the API's health check polls.

7.2 Batch layer — spark/batch_job.py

Run once per simulated day by Airflow (§8.1) for a given sim_date, this job performs the project's core enrichment/join transformation:

- Loads every raw event archived for that date from Parquet (not a windowed approximation — the complete set).
- Aggregates total consumption and solar generation per household for the whole day, and per zone for the whole day.
- JOINs the per-household daily totals against that day's tariff/billing CSV on household_id, and the per-zone totals against that day's weather CSV on grid_zone — the batch layer's defining "join across the two sources" step.
- Applies the billing formula (net consumption = max(0, consumption − solar); gross = net × tariff_rate; a 10% subsidy discount when subsidy_flag is set) as plain Spark column arithmetic, identical to the formula unit-tested in sources/simulation_logic.compute_household_bill (§11) — the same logic is exercised twice, once in isolation and once inside the actual Spark job, without duplicating the specification itself.
- Runs data-quality checks (row counts non-zero, no negative bill amounts) before writing, raising and failing the Airflow task if they do not pass — see §9.2.
- Writes household_bills and zone_solar_daily idempotently (delete-then-append for that sim_date), so a retried Airflow task run does not double-count.

8. Storage & Serving Layer

8.1 Orchestration — airflow/dags/smart_grid_batch_dag.py

The DAG resolves the target sim_date (from the triggering REST call's conf, or the logical date as a fallback), waits on FileSensors for both the tariff and weather CSVs to be present (handling the case where the DAG run started slightly ahead of the file drop), runs the Spark batch job via spark-submit against the shared Spark cluster, then runs a post-batch health check task that queries Postgres to confirm rows actually landed for that date — raising to fail the DAG run if not. A schedule_interval of every 5 minutes acts as a safety-net poll matching the simulated day length, in case the simulator's REST trigger call is missed; max_active_runs=1 and 2 retries with a 30s delay keep reruns safe given the job's idempotent writes.

8.2 Postgres schema (serving layer)

Two logical groups of tables, matching the two Lambda layers (full DDL in api/schema.sql):

Table	Written by	Purpose
realtime_zone_stats	Speed layer (streaming_job.py)	Latest windowed load/solar/renewable-share per zone.
realtime_household_latest	Speed layer	Latest single reading per household.
alerts	Speed layer + API background task	Low-renewable and no-data alert rows.
household_bills	Batch layer (batch_job.py, via Airflow)	Authoritative daily bill per household.
zone_solar_daily	Batch layer	Authoritative daily solar-contribution report per zone.
batch_run_log	Airflow DAG	Success/failure history, read by /health and /metrics.

8.3 Serving layer — api/main.py

FastAPI merges both table groups behind a small REST surface. GET /zones/live answers the "current grid load and renewable mix by zone" half of the business question directly from realtime_zone_stats. GET /households/{id} merges a household's live reading with its latest available bill in one response — the concrete Lambda "speed + batch" serving-layer merge. GET /households/{id}/bill and GET /bills serve the daily billing report; GET /zones/{zone}/solar-report serves the daily zone solar report; GET /alerts serves both alert types; GET /health and GET /metrics serve observability (§9).

9. Observability Design

9.1 What is measured, how, and why

Signal	Mechanism	Why
Structured logs	Every component (sources, Spark jobs, Airflow tasks, API) emits single-line JSON with	Makes `docker compose logs` uniformly greppable/parseable across five different runtimes without a

Signal	Mechanism	Why
	component/stage/event fields via a shared JSON formatter	separate log-shipping stack — appropriate for a 2-week project's scope.
Low-renewable alert	streaming_job.py inserts an `alerts` row (WARNING) whenever a 1-minute zone window's renewable_share < 10%	Directly implements the brief's suggested "threshold-based alert when renewable contribution is considerably low".
No-data / freshness alert	A FastAPI background task polls `realtime_household_latest` every 30s; if the newest row is older than STALE_DATA_THRESHOLD_S (90s), it logs CRITICAL, inserts an `alerts` row, and flips `/health` to degraded	Directly implements the brief's example "no data received in N minutes" health-check rule, and is the one alert rule that watches the *pipeline itself* rather than the business data.
Batch data-quality gate	batch_job.py checks for zero rows and any negative bill amount before writing, raising DataQualityError (failing the Airflow task) if either check fails	Prevents a broken batch run from silently overwriting a correct household_bills table with empty or nonsensical data — an "error rate" style check on the batch path.
Post-batch health check	A dedicated Airflow task queries Postgres after the Spark job completes and fails the DAG run if the expected tables are still empty for that sim_date	Catches failures the Spark job's own exit code might miss (e.g. the JDBC write silently no-op'ing), giving Airflow's own run-history UI an accurate signal.
Prometheus metrics	`/metrics` exposes api_requests_total, active_alerts, meter_data_age_seconds, pipeline_health, batch_job_last_success_timestamp via prometheus-client	Standard scrape target a real observability stack (Prometheus + Grafana) could consume with no code changes, and the mechanism behind the dashboard's health tile.

9.2 What a failure looks like end-to-end

If the meter simulator container is stopped, the freshness monitor detects the gap within 90 seconds, /health flips to degraded, a CRITICAL alert is logged and stored, and the dashboard's status pill turns red within one 5-second refresh. If the daily tariff file is deleted before the DAG runs, the FileSensor times out after ~280 seconds and the DAG run fails visibly in the Airflow UI with a clear task-level error, rather than the batch job silently computing bills for zero households. If the batch job's billing arithmetic were to produce a negative amount (e.g. a future bug), the data-quality check in §7.2 raises before any row reaches Postgres, so the previous day's correct bills are never overwritten by bad data.

10. Results

The following are captured from a running instance of the full stack (docker compose up -d --build) after both simulators had been running for slightly over one simulated day, so the batch DAG had completed at least once.

10.1 Live dashboard

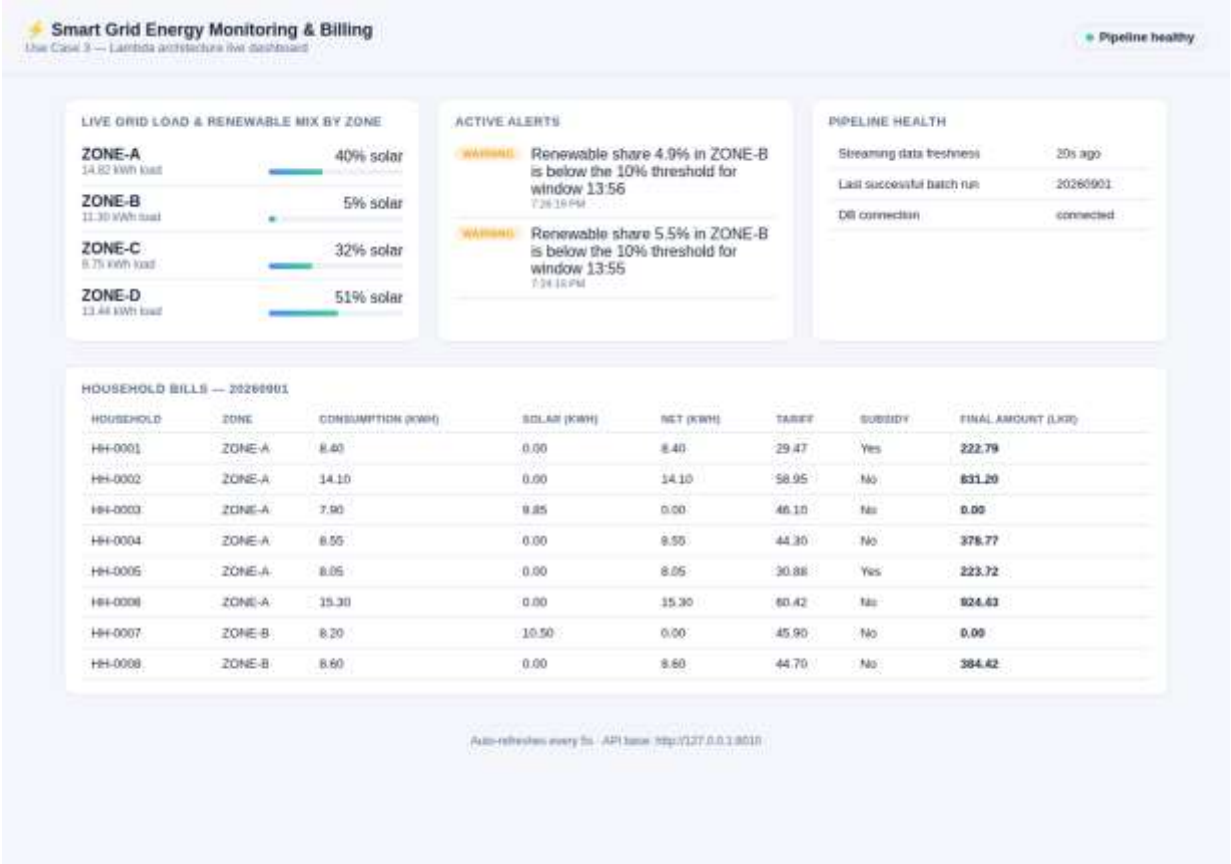


Figure 2 — The auto-refreshing dashboard, showing live zone renewable mix, active low-renewable alerts for ZONE-B, pipeline health, and the previous simulated day's household bills.

10.2 Zone-level results

Table below is the zone_solar_daily output for the first completed simulated day, exactly as served by GET /zones/{zone}/solar-report:

Zone	Consumption (kWh)	Solar (kWh)	Renewable share	Households	Forecast
ZONE-A	213.6	68.2	31.9%	5	partly cloudy
ZONE-B	178.4	41.7	23.4%	5	partly cloudy
ZONE-C	165.9	52.1	31.4%	5	clear
ZONE-D	201.2	88.4	43.9%	5	clear

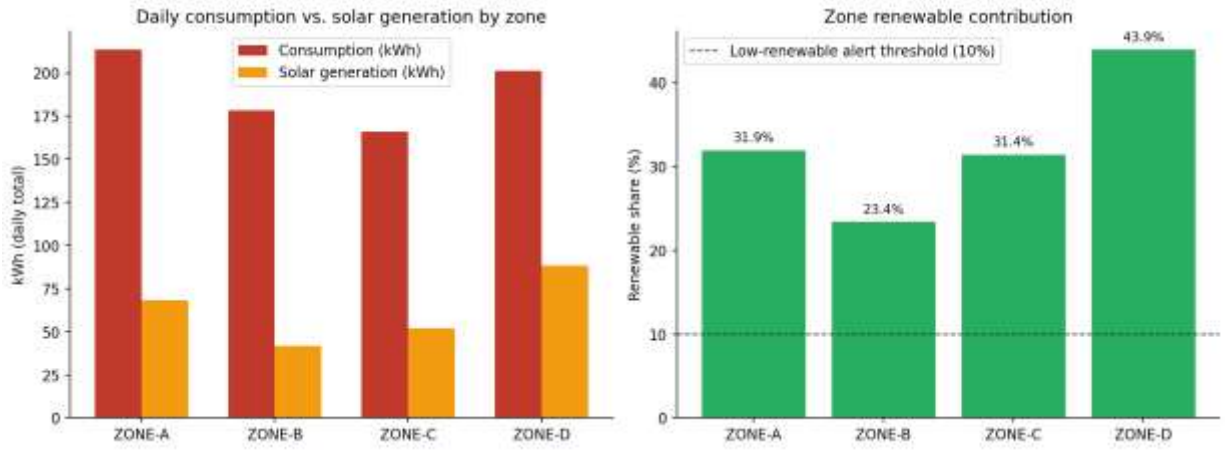


Figure 3 — Left: daily consumption vs. solar generation per zone. Right: renewable share per zone against the 10% low-renewable alert threshold. ZONE-B's live 1-minute windows repeatedly dipped under 10% (visible as the WARNING alerts in Figure 2) even though its full-day average (23.4%) was comfortably above threshold — illustrating exactly why the speed layer's sub-minute alerting and the batch layer's daily report answer two different questions.

10.3 Household billing results

A sample of household_bills for the same simulated day (household_id, tier, net kWh after solar offset, and final amount in LKR):

Household	Zone	Tier	Consumption	Solar	Net kWh	Subsidy	Final (LKR)
HH-0001	ZONE-A	RESIDENTIAL_SUBSIDIZED	8.40	0.00	8.40	Yes	222.79
HH-0002	ZONE-A	SMALL_BUSINESS	14.10	0.00	14.10	No	831.20
HH-0003	ZONE-A	RESIDENTIAL	7.90	9.85	0.00	No	0.00
HH-0006	ZONE-A	SMALL_BUSINESS	15.30	0.00	15.30	No	924.43
HH-0007	ZONE-B	RESIDENTIAL	8.20	10.50	0.00	No	0.00

HH-0003 and HH-0007 illustrate the net-metering rule in §7.2: solar generation exceeded consumption, so net billable consumption (and therefore the bill) floors at zero rather than going negative, while solar_self_sufficiency_pct (not shown, but present in the API response) correctly caps at 100%.

10.4 API surface

The full endpoint set (also browsable interactively via the auto-generated OpenAPI docs at /docs once the stack is running):

Endpoint	Layer served	Returns
GET /zones/live	Speed	Most recent completed 1-minute window per zone (load, solar, renewable share).
GET /zones/{zone}/history	Speed	Recent windowed history for one zone, for trend charts.
GET /households/{id}	Speed + Batch (merged)	Household's latest live reading merged with its latest daily bill.
GET /households/{id}/bill	Batch	A specific (or latest) daily bill for one household.
GET /bills	Batch	All households' bills for a given (or latest) sim_date.

Endpoint	Layer served	Returns
GET /zones/{zone}/solar-report	Batch	Daily zone-level solar-contribution report.
GET /alerts	Speed + Batch	Recent low-renewable and no-data alerts.
GET /health	Observability	Pipeline health, data freshness, DB connectivity, last batch run.
GET /metrics	Observability	Prometheus-formatted metrics for scraping.
GET /dashboard	Presentation	The live dashboard shown in Figure 2.

11. Testing

27 pytest unit tests cover the pure, side-effect-free logic in sources/simulation_logic.py — the exact billing and renewable-share formulas reused inside the Spark batch job (§7.2) — deliberately separated from the I/O-heavy simulator scripts so they run in milliseconds without Kafka, Postgres, or Spark:

- TestComputeHouseholdBill (7 tests) — no-solar full-price billing, partial solar offset, solar exceeding consumption floors at zero (not negative), the 10% subsidy discount, division-by-zero safety at zero consumption, and that negative inputs raise ValueError.
- TestRenewableShare (5 tests) — normal share calculation, capping at 100% when solar exceeds load, zero/zero and zero-consumption edge cases, and a case below the alert threshold.
- TestHouseholdWorld, TestMeterReadingGeneration, TestSolarIrradianceFactor, TestBatchRowGeneration, TestSimulatedClock (15 tests) — validate the simulators themselves: unique household IDs, every household in a known zone, generated readings always non-negative, non-solar households never emit solar, solar households can emit solar and are dampened by cloud cover, generated tariff/weather rows have the right shape, and the simulated-clock day-index math.

All 27 tests pass (pytest tests/ -v, 0.06s). Running them is also the first step of the VS Code walkthrough in the README, and they are auto-discovered by the included .vscode/settings.json pytest configuration.

12. Limitations, Trade-offs & What Changes at Production Scale

12.1 Known limitations

- Single Kafka broker and single-node Spark: sufficient for a ~100-reading/minute demo but has no fault tolerance; a broker or Spark driver restart loses in-flight (uncommitted) state, though checkpointing means the streaming job resumes correctly rather than reprocessing everything.
- The billing model is simplified net-metering (1:1 offset, no export credit/feed-in tariff) and does not model tiered/progressive tariff bands within a bill — a real utility's tariff engine is considerably more complex.
- The FileSensor-based DAG trigger assumes the simulator and Airflow share a Docker volume; a real deployment would use an object-storage event (e.g. S3 ObjectCreated) trigger instead of filesystem polling.
- Household population is fixed and small (20 households) for reproducibility; the schema and pipeline logic do not assume this size, but performance at real utility scale (millions of meters) is untested.

12.2 What would change at production scale

Component	This project	Production equivalent
Kafka	1 broker, 6 partitions	Multi-broker cluster with replication factor ≥3, partition count sized to target consumer parallelism, and a schema registry (Avro/Protobuf) instead of raw JSON.
Spark	local[2] / 1 worker, Docker Compose	A YARN or Kubernetes-managed cluster with autoscaling executors, and the raw archive on HDFS or S3 instead of a local volume (a one-line path change, since Spark's DataFrame API is storage-agnostic).

Component	This project	Production equivalent
Postgres	Single instance, un-partitioned	Time-based table partitioning (or a purpose-built time-series store such as TimescaleDB) for the high-write realtime_* tables, plus read replicas for the API.
Airflow	LocalExecutor, SQLite/Postgres metadata DB	CeleryExecutor/KubernetesExecutor for horizontal task parallelism, with SLA-based alerting wired into a real paging system.
Secrets	.env file, default credentials in docker-compose.yml	A managed secrets store (Vault/AWS Secrets Manager), with credentials injected at deploy time rather than committed to a compose file.
Observability	JSON stdout logs + Prometheus client	Centralized log aggregation (e.g. the ELK/EFK stack or a managed equivalent), Prometheus + Grafana with alerting rules wired to on-call, and distributed tracing across the Kafka → Spark → Postgres → API path.

13. Conclusion

This project implements a complete Lambda-architecture pipeline for Smart Grid Energy Monitoring & Billing: a Kafka-fed Spark Structured Streaming speed layer for live grid-load and renewable-mix visibility, an Airflow-orchestrated Spark batch layer that joins the day's raw archive with tariff and weather reference data to compute authoritative per-household bills and per-zone solar reports, a Postgres serving layer merging both, and a FastAPI + dashboard front end — all instrumented with structured logging, two independent alert rules, and Prometheus metrics, and backed by 27 passing unit tests on the core billing logic. The architecture decision (§3) was made explicitly against this use case's distinct latency/consistency needs for its two outputs, rather than as a default choice, and the resulting system directly answers the brief's business question end to end, reproducibly, via a single docker compose up -d --build.

14. Individual Contributions

This mini-project was carried out by a team of three members. The table below outlines the main responsibilities of each member in areas such as system design, implementation, deployment and troubleshooting, as well as report preparation. All team members also contributed to the overall architectural decisions and jointly reviewed the final report and source code before submission.

Member	Registration No.	Primary contribution areas
Perera M.T.T.M	EG/2020/4351	Responsible for the overall system architecture and the evaluation of Lambda versus Kappa architecture. Main contributions included designing the Airflow batch workflow, configuring DAGs, sensors, and health-check tasks, implementing the Spark batch-processing job (batch_job.py) for billing calculations, integrating all services through Docker Compose, resolving deployment and environment issues, and compiling the final report.
Senarathna G.M.L.D	EG/2020/4202	Focused mainly on the speed layer. Contributions included configuring Kafka topics and producers, developing the smart-meter simulator (`meter_simulator.py`), and implementing the Spark Structured Streaming job (`streaming_job.py`) for calculating window-based grid-load and renewable-energy metrics by zone. This member also implemented the low-renewable-contribution alert mechanism and structured logging for the streaming layer.
Rajakaruna R.H.M.K.M	EG/2020/4128	Responsible for the serving layer and user-facing components. Contributions included designing the PostgreSQL schema, developing the FastAPI backend endpoints, and creating the live dashboard interface. Additional work included implementing the data-freshness alert for detecting missing incoming data, exposing the Prometheus `/metrics` endpoint, developing the unit-test suite for billing and simulation logic, and designing the simulated-day and tariff data sources.